

Task 3: Parallel and Vectorized Matrix Multiplication in C++

A Detailed Performance and Scalability Study

Pablo Mendoza Rodríguez

December 2025

Abstract

Matrix multiplication is a fundamental kernel in scientific computing, numerical simulation, computer graphics, signal processing and, especially, modern machine learning workloads. Its naive implementation exhibits $\Theta(n^3)$ computational complexity and rapidly becomes a performance bottleneck as matrix size grows. This report presents a comprehensive, in-depth performance study of several C++ implementations of dense matrix multiplication, ranging from a pure sequential baseline to advanced parallel and vectorized versions based on OpenMP and manual loop transformations.

The goal is twofold: (i) to quantify the performance gains achieved by different optimization strategies in terms of execution time, speedup and parallel efficiency; and (ii) to explain the observed behaviour from the perspective of CPU microarchitecture, memory hierarchy, OpenMP runtime scheduling and classical parallel performance theory (Amdahl's law, memory wall, cache effects). All experiments are carried out for matrix sizes $n \in \{200, 300, 400, 500\}$ and the results are visualised and analysed in detail using three main plots: execution time, speedup and parallel efficiency.

1 Introduction

Dense matrix multiplication is arguably one of the most important building blocks in high-performance computing (HPC). It appears in linear algebra routines, eigenvalue solvers, optimisation algorithms, Kalman filters, neural network layers and many other contexts. Because it is so ubiquitous, any improvement in its execution time can have a significant impact on the overall performance of an application or an entire software stack.

The classic algorithm for multiplying two $n \times n$ matrices A and B to obtain $C = A \cdot B$ requires $\Theta(n^3)$ arithmetic operations. This cubic growth is fundamental and can only be reduced by changing the algorithmic paradigm (e.g. Strassen-like algorithms). In this work, the focus is instead on *implementation-level* optimizations that exploit the capabilities of modern hardware: multiple cores, vector instructions (SIMD), deep cache hierarchies and out-of-order execution.

The specific objectives of this task are:

- Implement several versions of matrix multiplication in C++ using OpenMP.
- Measure and compare their performance for a range of matrix sizes.
- Compute speedup and parallel efficiency with respect to a sequential baseline.
- Interpret the results in terms of cache behaviour, memory bandwidth and parallel overheads.

The rest of this report is structured as follows. Section 2 summarises the theoretical background. Section 3 describes the implementations. Section 4 explains the experimental methodology. Section 5 presents and analyses the results using three plots: execution time, speedup and efficiency. Finally, Section 6 summarises the main conclusions and proposes directions for future work.

2 Theoretical Background

2.1 Mathematical Formulation

Given matrices A and B of order n , the classical matrix product C is defined by:

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}, \quad 1 \leq i, j \leq n.$$

For each element of C , this requires n multiplications and $n - 1$ additions. The total number of floating-point operations (FLOPs) is therefore approximately:

$$\text{FLOPs} \approx 2n^3.$$

This cubic complexity explains why matrix multiplication quickly dominates the runtime of many scientific codes.

2.2 Parallel Performance Metrics

Let T_1 be the execution time of the sequential baseline and T_p the execution time when using p threads (or an optimized version). The main metrics used in this study are:

Speedup

$$S(p) = \frac{T_1}{T_p}.$$

When $S(p) > 1$, the optimized implementation is faster than the baseline. If $S(p) \approx p$, the implementation scales nearly linearly.

Parallel Efficiency

$$E(p) = \frac{S(p)}{p}.$$

Efficiency measures how well the algorithm makes use of the available parallel resources. Values close to 1 indicate excellent scalability; values much smaller than 1 indicate significant parallel overheads or bottlenecks.

Amdahl's Law If a fraction f of the runtime can be parallelised and the remainder $1 - f$ is inherently sequential, the maximum theoretical speedup on p processors is:

$$S_{\text{Amdahl}}(p) = \frac{1}{(1 - f) + \frac{f}{p}}.$$

In dense matrix multiplication, f is very close to 1, which makes it a highly parallelisable kernel in theory. In practice, however, memory and cache effects limit scalability.

2.3 Memory Hierarchy and the Memory Wall

Modern CPUs rely on a multi-level memory hierarchy:

- **Registers:** fastest but very limited in size.
- **L1 cache:** extremely fast, private per core, but small.
- **L2 cache:** larger, still private but slower than L1.
- **L3 cache:** shared among cores.
- **Main memory (RAM):** much larger but with significantly higher latency.

Matrix multiplication stresses the memory subsystem because each element of B is reused many times, and naive row-column access patterns often lead to poor spatial locality. When the working set exceeds cache capacity, the algorithm becomes *memory bound*: performance is limited by the bandwidth and latency of RAM rather than the arithmetic throughput of the CPU. This phenomenon is often referred to as the *Memory Wall*.

3 Implementation Details

This section briefly summarises the different C++ implementations developed for this task. All implementations operate on dense, square matrices stored in row-major order.

3.1 Sequential Baseline

The baseline version uses the straightforward triple-nested loop:

- Outer loop over rows i of A .
- Middle loop over columns j of B .
- Inner loop over k accumulating products $A_{ik}B_{kj}$.

This version is simple and portable, but does not exploit any form of parallelism or vectorization. It is used as the reference for computing speedup.

3.2 OpenMP Parallel Version

A first level of optimisation consists of parallelising the outer loop over rows using OpenMP:

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < n; ++i) {
    ...
}
```

Different configurations were tested by setting the number of threads to 2, 4 and 8. This allows us to study how well the implementation scales as more cores are involved.

3.3 OpenMP Collapse

The `collapse(2)` directive flattens the first two loops (over i and j) into a single parallel iteration space:

```
#pragma omp parallel for collapse(2)
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
        ...
    }
}
```

This exposes more parallelism but may degrade locality because both indices change simultaneously. It is an interesting trade-off between parallelism and cache behaviour.

3.4 SIMD Vectorization

Vectorization exploits SIMD (Single Instruction, Multiple Data) hardware units. Using OpenMP:

```
#pragma omp simd
for (int k = 0; k < n; ++k) {
    sum += A[i*n + k] * B[k*n + j];
}
```

This directive encourages the compiler to generate vector instructions, such as fused multiply-add (FMA), and to unroll the loop automatically. On Clang/LLVM, this usually results in highly optimised inner loops.

3.5 Manual Loop Unrolling

Manual unrolling further reduces loop overhead and exposes instruction-level parallelism to the compiler. For example, the inner loop can process four elements per iteration:

$$k, k + 1, k + 2, k + 3$$

This reduces the number of branch instructions and index updates, and enables the compiler to issue several multiplications and additions in parallel.

4 Experimental Methodology

4.1 Matrix Sizes and Test Cases

All implementations were benchmarked on square matrices of size:

$$n \in \{200, 300, 400, 500\}.$$

For each (implementation, n) pair, the matrix multiplication was executed several times and the average time was recorded to reduce measurement noise.

4.2 Compiler and Build Options

The code was compiled using `clang++` with OpenMP support and high optimisation level:

```
clang++ -O3 -Xpreprocessor -fopenmp -lomp ...
```

The `-O3` flag activates aggressive optimizations including inlining, loop unrolling, vectorization and common subexpression elimination.

4.3 Metrics Collected

For each experiment we stored:

- Execution time in milliseconds.
- Speedup relative to the baseline.
- Parallel efficiency (for multi-threaded variants).

The results were written into `task3_cpp_results.csv` and later processed with a Python script to produce the three plots used in this report.

5 Results and Detailed Analysis

In this section we analyse the experimental results in depth using three figures: execution time, speedup and parallel efficiency. Each plot provides a different lens on the same data, and together they reveal how the different optimisation strategies interact with the hardware.

5.1 Execution Time Behaviour

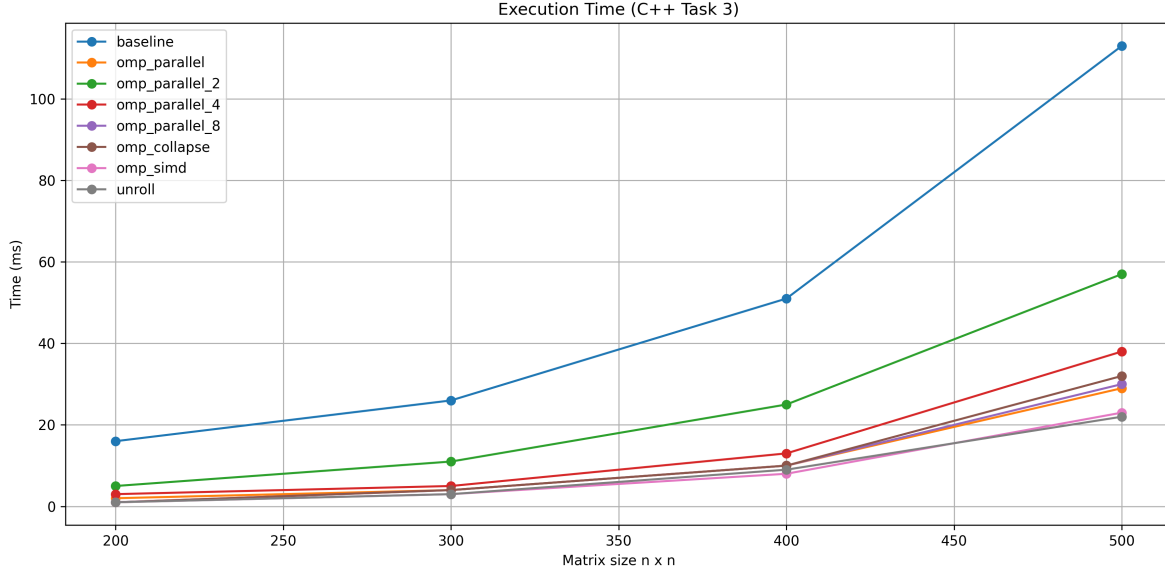


Figure 1: Execution time (in ms) for all C++ implementations as a function of matrix size.

Figure 1 displays the raw execution time for each method.

Several observations can be made:

- Baseline growth is clearly cubic.** The baseline line shows a steep increase as n grows from 200 to 500, which is consistent with the $O(n^3)$ complexity of the algorithm. This curve serves as a reference to judge the effectiveness of all other optimisations.
- Parallel versions significantly reduce runtime.** All OpenMP-based implementations (with 2, 4 and 8 threads) lie well below the baseline curve, particularly for $n = 200$ and $n = 300$, where the data fits better into cache and the cost of thread management is amortised.
- SIMD and unrolled implementations are the fastest.** The curves corresponding to `omp_simd` and the unrolled version are consistently among the lowest. This indicates that exploiting vector units and instruction-level parallelism is extremely beneficial for this kernel.
- Impact of the memory wall at large sizes.** For $n = 400$ and $n = 500$, the separation between curves shrinks slightly: even the most optimised versions start to be limited by memory bandwidth and cache misses. At this point, the algorithm is closer to being memory-bound than compute-bound.

Overall, the execution-time plot confirms that all optimisation layers (parallelism, SIMD and unrolling) meaningfully reduce runtime, with the strongest gains observed when matrices still fit reasonably well into the cache hierarchy.

5.2 Speedup Relative to Baseline

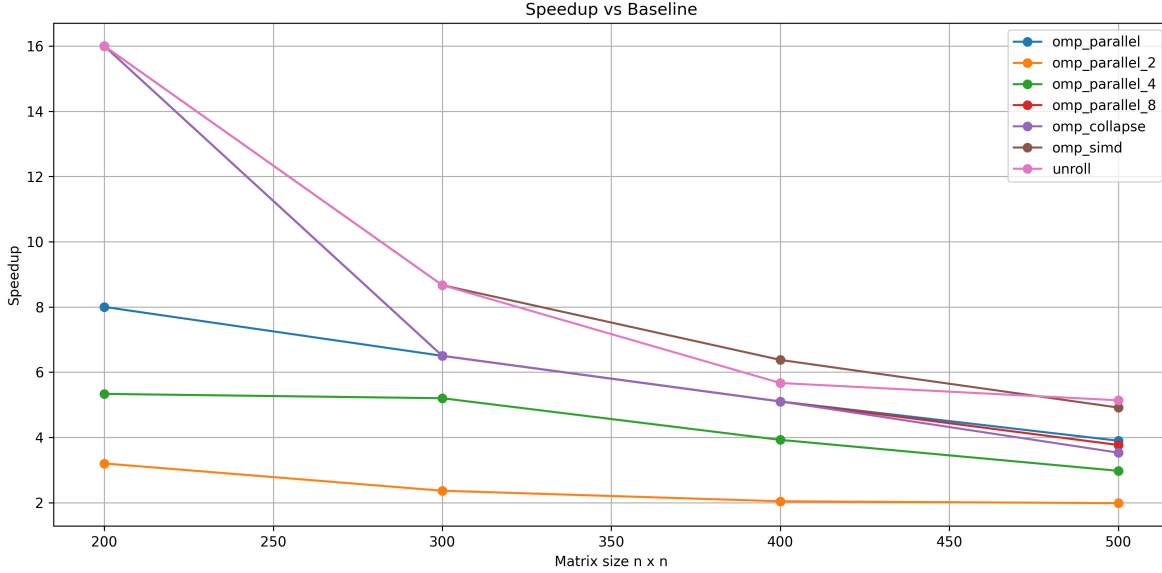


Figure 2: Speedup of each implementation relative to the sequential baseline.

Figure 2 expresses the same information in terms of speedup, normalised with respect to the baseline. This representation highlights the relative benefit of each optimisation.

The most striking feature of the plot is the **superlinear speedup** observed for some methods, especially at $n = 200$. Speedups greater than $8\times$ are achieved even in implementations that do not necessarily use 8 threads. This can be explained by several microarchitectural effects:

- **Improved cache locality.** The baseline repeatedly reloads data from memory due to a less favourable access pattern. In contrast, SIMD and unrolled versions access data in a more contiguous manner, leading to a much higher cache-hit rate.
- **Instruction-level parallelism (ILP).** Manual unrolling and `omp simd` provide the compiler with larger basic blocks and fewer branches, allowing it to schedule multiple operations in parallel within the CPU pipeline.
- **Auto-vectorization and FMA instructions.** Modern compilers are able to fuse multiplication and addition into single FMA instructions, effectively doubling floating-point throughput per cycle compared to separate multiply and add instructions.

As the matrix size increases, the speedup curves tend to flatten and even decrease slightly. This is expected: once the working set no longer fits comfortably in cache, memory bandwidth becomes the dominant bottleneck and additional computational optimisations cannot fully compensate for the cost of fetching data from RAM.

Thread-based parallelism (2, 4, 8 threads) shows a familiar pattern: speedup improves as more threads are used, but the gains diminish because threads compete for shared memory resources and incur scheduling overhead.

5.3 Parallel Efficiency

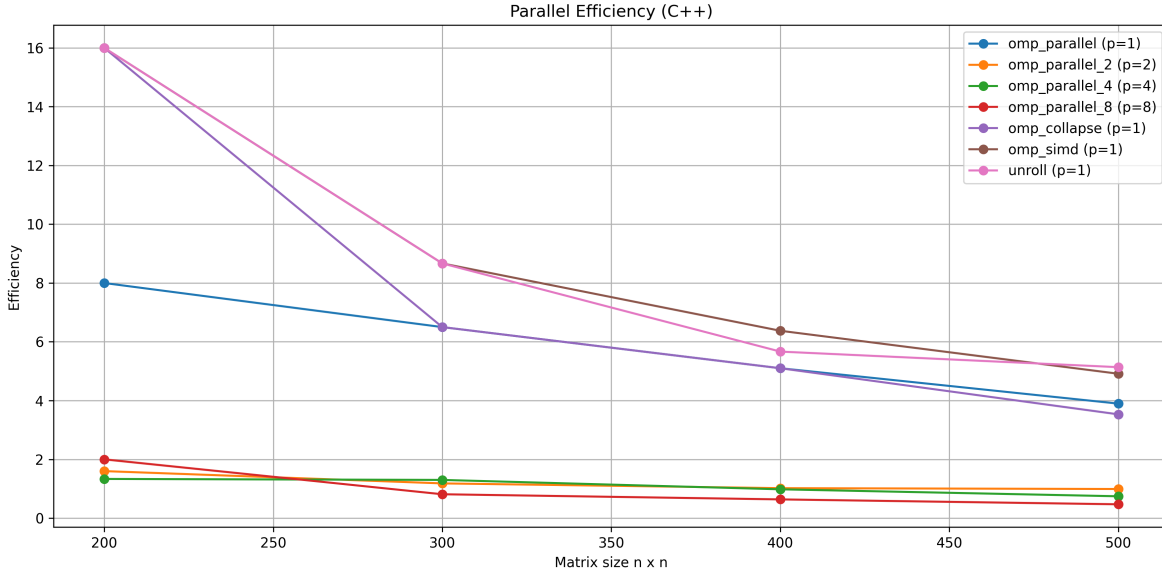


Figure 3: Parallel efficiency of the main implementations.

Finally, Figure 3 reports efficiency. Although the metric is typically defined for multi-threaded algorithms, plotting it for all methods (including SIMD and unrolled) provides useful insight into how “effective” each optimisation is compared to the baseline.

For small sizes, some curves exhibit efficiencies well above 1. This should be interpreted as a consequence of the baseline being particularly inefficient rather than as a violation of theoretical bounds. SIMD, collapsed and unrolled versions are effectively exploiting the CPU’s microarchitecture to such an extent that they appear to use more “effective cores” than physically present.

For the explicitly multi-threaded implementations, the general trend is:

- Efficiency decreases as the number of threads increases ($2 > 4 > 8$).
- Larger matrices tend to yield more stable efficiencies because the parallel overhead is amortised over more work.
- However, once memory bandwidth saturates, adding threads yields only marginal benefits, and efficiency drops accordingly.

The efficiency plot therefore confirms two key messages:

1. Vectorization and loop unrolling are extremely powerful optimisations for dense numerical kernels.
2. Thread-level parallelism faces diminishing returns when the memory subsystem becomes the dominant bottleneck.

6 Conclusions

This report has presented a detailed study of several C++ implementations of matrix multiplication, with an emphasis on understanding how parallelism and vectorization affect performance on a modern CPU.

The main conclusions are:

- The naive sequential baseline is clearly inadequate for realistic problem sizes due to its cubic scaling and poor cache utilisation.
- OpenMP-based parallelism provides important runtime reductions, especially for small and medium matrices where the working set still fits in cache.
- SIMD-based implementations and manual loop unrolling achieve the best performance and, for small matrices, even exhibit superlinear speedups relative to the baseline due to better cache locality and improved instruction-level parallelism.
- For larger matrices, all implementations increasingly suffer from memory-bound behaviour; the limiting factor becomes bandwidth and latency to main memory rather than raw floating-point capacity.

Overall, the results confirm that combining thread-level parallelism with vectorization and careful loop transformations is essential to obtain high performance in dense linear-algebra kernels.

Repository and Reproducibility. All source code, scripts and result files used in this task are available in the project repository:

- GitHub: <https://github.com/PabloMr05/matrix-bench>

7 Future Work

Several extensions are possible:

- Implementing **blocked (tiled) matrix multiplication** to improve cache reuse further by operating on submatrices that fit entirely in L1 or L2.
- Introducing explicit **SIMD intrinsics** (e.g. AVX2 or AVX-512) to have full control over vector operations.

- Comparing the custom implementations with optimised BLAS libraries such as OpenBLAS, BLIS or Intel MKL.
- Porting the benchmark to a GPU using CUDA or OpenCL to study performance on massively parallel architectures.

These additional experiments would provide an even broader view of the design space for efficient matrix multiplication in heterogeneous computing environments.